

UNIVERSIDADE MOGI DAS CRUZES - UMC
CURSO DE BACHARELADO EM SISTEMAS DE INFORMAÇÃO

JOSÉ ARMANDO ABRÃO BOER

RELATÓRIO TÉCNICO: SISTEMA DE GERENCIAMENTO DE INGRESSOS
MODELAGEM E IMPLEMENTAÇÃO COM HERANÇA E POLIMORFISMO EM
JAVA WEB

1. INTRODUÇÃO

O presente relatório descreve as decisões arquiteturais e de modelagem adotadas para o desenvolvimento de um Sistema de Gerenciamento de Ingressos. O objetivo principal do projeto foi construir uma aplicação web robusta capaz de gerenciar diferentes categorias de ingressos (Normal, VIP e Meia-entrada), permitindo o cadastro, cálculo de valores dinâmicos, autenticação de usuários e fluxos de cancelamento e devolução. O projeto foi desenvolvido utilizando a linguagem Java com o framework Spring Boot e o banco de dados NoSQL MongoDB.

2. DECISÕES DE MODELAGEM E ARQUITETURA

Para garantir a escalabilidade e a fácil manutenção do código, o sistema foi desenhado utilizando o padrão de arquitetura em multicamadas (N-Tier Architecture), amplamente adotado no ecossistema Spring:

- **Camada de Modelo (Model):** Responsável por representar as entidades de negócio e suas regras fundamentais, incluindo a máquina de estados do ingresso (Disponível, Pago, Emitido, Cancelado, Devolvido).
- **Camada de Repositório (Repository):** Interface responsável pela persistência dos dados. Optou-se pelo uso do Spring Data MongoDB, que simplifica as operações de CRUD e permite consultas personalizadas, como a filtragem de ingressos por usuário logado.
- **Camada de Serviço (Service):** Centraliza as regras de negócio complexas, como a validação de políticas de cancelamento e devolução (verificação de datas e permissões de propriedade do ingresso), isolando essa responsabilidade dos controladores.
- **Camada Controladora (Controller):** Expõe os endpoints da API REST, recebendo as requisições HTTP do front-end e delegando o processamento à camada de serviço.

A interface de usuário foi implementada no padrão Single Page Application (SPA) simplificado, utilizando HTML5, Bootstrap e JavaScript puro (Fetch API) consumindo a API REST do back-end de forma assíncrona.

3. IMPLEMENTAÇÃO DE HERANÇA E POLIMORFISMO

O núcleo técnico deste projeto reside na aplicação correta dos pilares da Orientação a Objetos para tratar diferentes tipos de ingressos de forma unificada.

3.1. HERANÇA

Foi criada uma classe abstrata denominada Ingresso, que atua como a superclasse (mãe). Esta classe encapsula todos os atributos comuns a qualquer ingresso, como id, evento, dataEvento, valorBase, estado e o usuarioid.

A partir desta superclasse, foram implementadas três subclasses (filhas):

- **IngressoNormal:** Não altera o valor base.
- **IngressoVIP:** Adiciona o atributo específico taxaVIP.
- **IngressoMeia:** Adiciona o atributo específico percentualDesconto.

A classe abstrata Ingresso define as assinaturas dos métodos calcularValor() e imprimirIngresso(), obrigando as subclasses a sobrescrevê-los (@Override) para implementar seus comportamentos específicos de cálculo financeiro.

3.2. POLIMORFISMO NO CÓDIGO E NA WEB (JSON)

O polimorfismo foi aplicado na camada de Serviço e no Controlador. O método salvar() e as buscas no repositório trafegam objetos do tipo genérico Ingresso. Em tempo de execução, a Máquina Virtual Java (JVM) invoca o método calcularValor() correto dependendo da instância concreta (Normal, VIP ou Meia) que está alocada na memória.

Como o sistema é uma API Web que trafega dados em formato JSON, foi necessário garantir que o framework Jackson (integrado ao Spring) compreendesse o polimorfismo durante a conversão (deserialização). Para isso, utilizamos as anotações @JsonTypeInfo e @JsonSubTypes na superclasse. Ao receber a propriedade "tipo": "VIP" via requisição HTTP POST do front-end, o sistema dinamicamente instancia um objeto IngressoVIP, demonstrando o polimorfismo prático desde a requisição web até a persistência no MongoDB, que armazena a referência da classe concreta de forma transparente.

4. CONCLUSÃO

A modelagem adotada cumpriu integralmente os requisitos do diagrama de classes e da máquina de estados. O uso de herança evitou a duplicação de código, centralizando atributos comuns, enquanto o polimorfismo permitiu que o sistema fosse programado para a interface da classe abstrata. Esta abordagem torna o sistema altamente extensível; caso haja a necessidade de criar um novo tipo de ingresso (ex: "Ingresso Cortesia"), basta criar uma nova classe filha, sem necessidade de modificar a lógica de persistência ou o roteamento do controlador principal.